

With the following command, you can see all the commands that were run with an image via a container:

```
$ docker history ImageID
```

Dockerfile

This is one of the most important topics. To familiarise oneself with a Dockerfile, look at the following example:

```
#This is a sample Image
FROM ubuntu
MAINTAINER demousr@gmail.com
RUN apt-get update
RUN apt-get install -y nginx
CMD ["echo","Image created"]
```

Let us now look at how the Dockerfile is written and use it according to requirements with a simple *hello world* application in PHP:

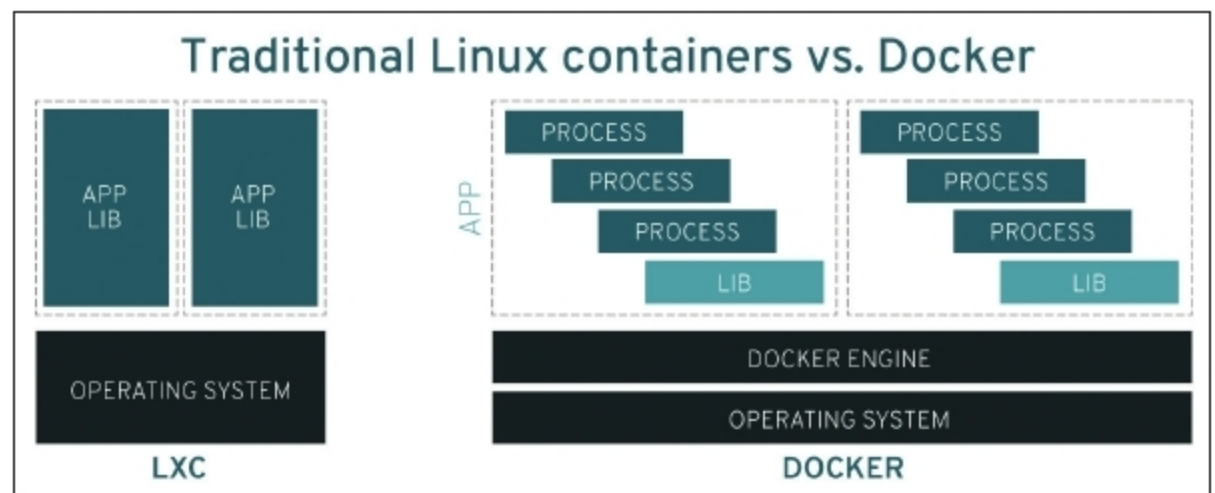
```
//index.php
<?php
echo "Hello World";
```

To execute this file and host it, you need a Web server. You can create one using Docker and configure the environment. To host a Web server, you will need Apache. You don't have to start from scratch because you can create a file with an existing Docker image. You can find the existing images in Docker Hub.

As you can see, there is a long list of images with the name *php*. Select the one that you require and mention it in the Dockerfile.

```
//Dockerfile
FROM php:7.0-apache
COPY src/ /var/www/html
EXPOSE 80
```

Now, in the folder where the Dockerfile is present, run the command, and the output you will get is as follows. The *build* command is used when you want to create your own Docker images.



Source: <https://www.redhat.com/en/topics/containers/what-is-docker>

```
$ docker build -t hello-world .
```

Since you have successfully completed building the image, let us now test it.

```
$ docker run -p 80:80 hello-world
```

You need this *-p* tag to forward a Port 80 from the host to Port 80 in the container, to get the desired output. But when you change the code in *index.php*, the change won't be reflected in the new version of the file because when you built the image, it made a copy of the file you wrote. This is where we require volumes.

This concludes the basic concepts for using Docker. But before I end, let me answer two common questions with respect to Docker, to dispel any misconceptions about it.

Is Docker technology the same as traditional Linux containers?

No, it's not the same. The Docker container technology was initially based

on Linux containers technology. Linux containers were useful for lightweight virtualisation, but didn't offer a great user experience. Docker technology not only has the capability to run containers and deploy software and applications, but also simplifies the process of creating and building containers, shipping images, and versioning of images.

What is the difference between virtual machines (VMs) and containers?

In a nutshell, a VM provides an abstract machine that uses device drivers targeting this abstract machine, while a container provides an abstract OS. The most important difference between the two is that containers provide ways to virtualise an OS; hence, multiple workloads can run on a single OS instance. With virtual machines, on the other hand, the hardware is also virtualised to run multiple OS instances. Compared to VMs, containers have more speed, agility and portability. **END** 🐧

References

- [1] <https://www.tutorialspoint.com/docker>
- [2] <https://acw-distribution.com.hk/en/solution-detail.php?id=26>
- [3] <https://github.com/collabnix/dockerlabs/blob/master/beginners/docker/what-is-docker.md>
- [4] <https://www.electronicdesign.com/technologies/dev-tools/article/21801722/whats-the-difference-between-containers-and-virtual-machines>

By: Bhanu Prakash Poluparthi

The author is an open source enthusiast and has been a part of the Drupal organisation since 2017. He was an intern at Google Summer of Code 2017 and a mentor at Google Code-In 2018.